WHITEPAPER

Fabric Apps: Bridging Data, Analytics and AI in a Single Ecosystem

A case study: building an AI-assisted Fabric App with write-back on the Interactive Sales Dashboard semantic model

Author: U Giri Shankar
Date: 18 October 2026
Workspace: BI Reports (Microsoft Fabric)
Source model: Interactive sales dashboard (Power BI semantic model)
Tooling: Claude Code in VS Code · Rayfin CLI 1.36.2 · React + Vite · Fabric SQL database

Contents

1. Executive summary 4
2. Starting point: the Interactive Sales Dashboard 5
    2.1 Semantic model used by the app 5
    2.2 What the report cannot do 6
3. Solution overview 7
    3.1 The four sections 8
4. Before you start: prerequisites and tools 9
    4.1 Prerequisites 9
    4.2 Tools used 9
    4.3 Who does what: developer and Claude 9
5. Build the app with Claude in VS Code 10
    5.1 Step 1: create the Fabric App item in the portal 10
    5.2 Step 2: open a project folder in VS Code 10
    5.3 Step 3: add the Claude Code extension 10
    5.4 Step 4: give Claude the requirement prompt 10
    5.5 Step 5: Claude builds, tests and deploys 12
6. Commands Claude ran: from connection to deployment 14
    6.1 Scaffold the project 14
    6.2 Add capabilities 14
    6.3 Connect the semantic model 14
    6.4 Validate DAX before writing UI 14
    6.5 Build and test the app 15
    6.6 Deploy into the existing app item 16
    6.7 Run locally (optional, after the first deployment) 17
    6.8 Issues Claude met and tips 17
    6.9 Command summary 18
7. Feature walkthrough: the finished app 20
    7.1 Pulse — live performance vs plan 20
    7.2 Plan Studio — collaborative forecast (write-back) 20
    7.3 Scenario Lab — what-if on real economics 21
    7.4 Action Board — turn insights into actions 22
8. Write-back use case: collaborative sales forecasting 24
    8.1 Business scenario 24
    8.2 Step by step 24
    8.3 Data model 26
    8.4 Security and governance 27
9. Validation 29
10. Differentiators vs the Power BI Interactive Sales Dashboard 30
11. Operating and extending the app 31
    11.1 Day-to-day use 31
    11.2 Future enhancements 31
12. Advantages and technical limitations of the Fabric App approach 32
    12.1 Advantages 32
    12.2 Technical limitations 32
13. Conclusion 34
14. References and acknowledgements 35
    14.1 Dataset credit 35
    14.2 Original work in this project 35

14.3 Technical references..... 35
Appendix A. Reference identifiers..... 36
Appendix B. Project structure..... 36
Appendix C. DAX queries used by the app..... 36

1. Executive summary

The **Interactive Sales Dashboard** is a Power BI report that answers *what happened*: total sales, cost, profit and year-over-year change by region, country, product and channel, plus customer feedback. It is excellent for reading the numbers, but like every Power BI report it is **read-only**: nobody can record what they intend to do about those numbers.

The **Sales Dashboard App** is a Microsoft Fabric App built with the Rayfin framework. It reuses exactly the same semantic model through a live connector, so every figure matches the report to the cent, and adds what a report cannot: people can **type plans, save scenarios and assign actions**, and those inputs are written to a Fabric SQL database owned by the app and shared instantly with the team.

The headline use case is **collaborative sales forecasting with write-back**. Each planner sees the latest actuals by region and product category (live DAX), enters growth assumptions for next year, saves a draft and submits it. The app stores every planner's numbers, computes a team consensus, and shows who changed what and when. Two further capabilities complete an *insight → plan → action* loop: a **Scenario Lab** for what-if pricing and cost modelling, and an **Action Board** that turns signals detected in the model (declining markets, loss-making products, 1–2★ orders) into owned, dated follow-ups.

> **Outcome**
>
> Built, tested (61 automated tests passing) and deployed to the **BI Reports** workspace with `npx rayfin up`. Write-back verified end to end: values typed in the app appear in the `dbo.SalesPlans` and `dbo.ActionItems` tables within seconds (Figures 13 and 14).

> **How to read this whitepaper**
>
> **Why and what (Sections 2–3):** the existing report, what it cannot do, and the solution design. **Build path (Sections 4–6):** prepare the tools, then build the app with Claude in VS Code in five steps; Section 6 lists every command Claude ran. Follow these sections to build your own app. **Result (Sections 7–9):** the finished app, the write-back use case and how it was validated. **Evaluation (Sections 10–13):** comparison with Power BI, operation, advantages and limitations, and conclusion.

	Interactive Sales Dashboard (Power BI)	Sales Dashboard App (Fabric App)
Primary question	What happened?	What will we do about it, and who owns it?
Data	Semantic model (import)	Same semantic model (live DAX) + app-owned SQL tables
User input	Slicers and filters only	Forecasts, notes, scenarios, actions — saved and shared
Output	Visuals	Visuals + decisions recorded in a database

2. Starting point: the Interactive Sales Dashboard

The existing report is published in the BI Reports workspace and is backed by the **Interactive sales dashboard** semantic model. It has four pages: **Sales Overview**, **Sales Insight**, **Customer Feedback** and **Conclusions**.

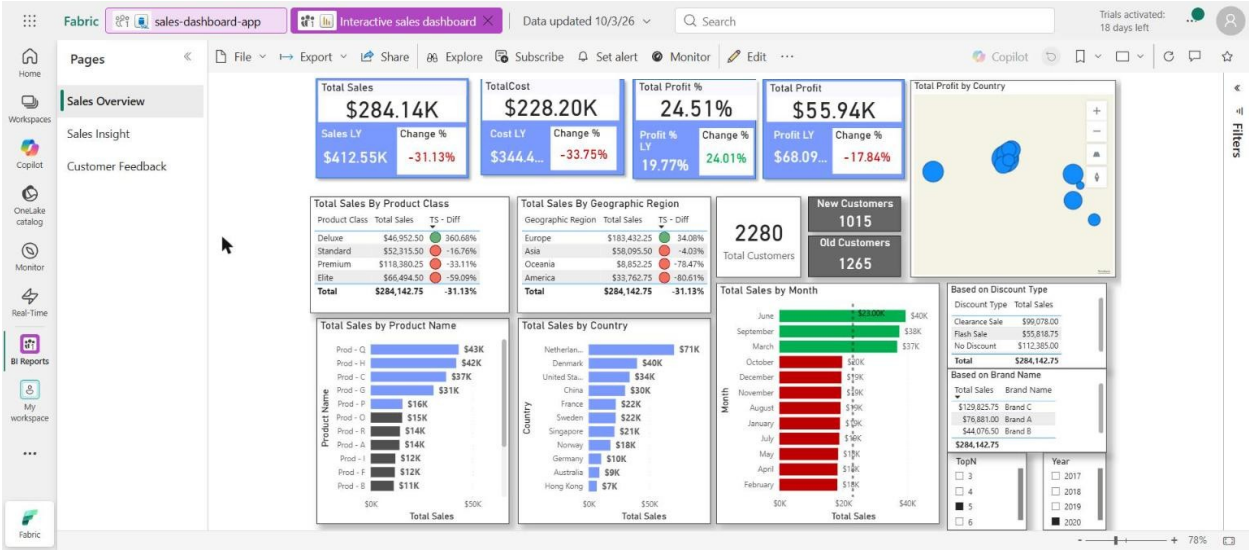


Figure 1. *The existing Power BI report (Sales Overview, 2020). Note the totals: $284.14K sales, $55.94K profit, −31.13% vs last year, 2,280 customers.*

2.1 Semantic model used by the app

Table	Role	Key columns / measures used
Sales Data	Fact (8,044 order lines, 2017–2020)	Sales Date, Quantity, Cost Price, Sales Price WO Disc, Discount, Final Sales Price, Profit/Loss, Customer Rating, Complaints Reason
Calender	Date dimension (calculated)	Year, Month, Month No
Location Details	Geography	Country, Geographic Region (America, Asia, Europe, Oceania)
Products Details	Product	Product Name, Products Category (8 shoe categories), Product Class
Sales Target	Monthly targets	Target
All Measures	Measure table	[Total Sales], [Total Cost], [Total Profit], [Total Profit %], [Sales LY], [Profit LY], [Total Customers], [New Customer], [Total Target Sales], [Customer Rating Count]

Model facts confirmed through the connector before any code was written: total sales **$1,088,726**, profit **$205,506**; sales by year 2017 $165,797 · 2018 $226,237 · 2019 $412,549 · 2020 $284,143 (−31% year over year).

2.2 What the report cannot do

- It cannot capture input. There is no supported way for a viewer to type a forecast, a target adjustment or a comment and have it stored.
- It cannot run multi-step workflows such as *draft → submit*, or *open → in progress → done*.
- What-if parameters exist, but the chosen values are not saved, named, shared or compared across users.
- It cannot attribute a change to a person and time (an audit trail of decisions).

3. Solution overview

The app is a single Fabric App item (`sales-dashboard-app`) in the BI Reports workspace. Users sign in with Microsoft Entra ID, either inside the Fabric portal or directly in a browser. The React frontend talks to two data sources:

- **Semantic model (read).** DAX queries run through the `fabric-semanticmodel` connector named `salesmodel`, using *delegated* authentication, so every user sees the data their own Power BI permissions (including any row-level security) allow.
- **App SQL database (read and write).** Three tables owned by the app (`SalesPlans`, `Scenarios`, `ActionItems`) are created from TypeScript entity definitions and exposed through Rayfin's generated GraphQL data API.

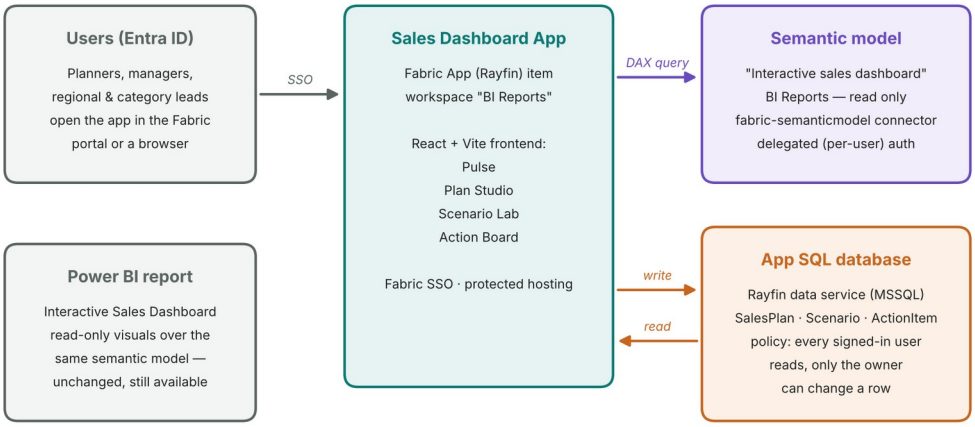


Figure 2. *Architecture of the Sales Dashboard App.*

Reading the diagram.

- **Purple arrow (DAX query):** the arrow shows the direction in which the query is sent. The app sends a DAX query, for example sales by region for 2020. The semantic model evaluates it with its own measures and returns a table of results, which the app displays. The model is only read, never changed.
- **Orange arrows (write and read):** when a user saves a plan, scenario or action, it is written to the app SQL database. The app then reads those rows back to show team averages, saved scenarios and the action board. Every signed-in user can read all rows, but only the person who created a row can update or delete it. This rule is enforced by the server, not just by the screen.
- **Power BI report (no arrows):** the existing Interactive Sales Dashboard report is unchanged and continues to read the same semantic model. The app neither replaces it nor depends on it.
- **Bringing it together:** each page combines existing data from the semantic model with user input from the SQL database. For example, Plan Studio shows the actual sales for each region and category next to each planner's growth % and the team average.

3.1 The four sections

Section	Purpose	Reads from model	Writes to app database
Pulse	Live KPIs and monthly trend vs last year and model target; team plan and open actions at a glance	Yearly KPIs, monthly trend	—
Plan Studio	Collaborative forecast for next year by region × category (write-back use case)	Region × category actuals	SalesPlans
Scenario Lab	What-if price, discount, volume, elasticity and cost levers on real category economics	Category sales, cost, quantity, discount	Scenarios
Action Board	Signals detected in the model become owned, dated actions on a board	Declining countries, loss-making products, 1–2★ orders	ActionItems

4. Before you start: prerequisites and tools

The app is built in five simple steps (Section 5): create the app item in the Fabric portal, open a folder in Visual Studio Code, add the Claude Code extension, give Claude the requirement prompt, and let Claude build, test and deploy the app. This section lists what must be in place first.

4.1 Prerequisites

- A Fabric workspace (**BI Reports**) on a capacity with the Fabric Apps workload enabled, and Contributor (or higher) rights in it.
- The semantic model **Interactive sales dashboard** published to the Power BI Service (local `.pbix` files cannot be used as a source).
- Node.js 24 and npm 11 installed on the PC; Visual Studio Code; a Claude account.

4.2 Tools used

Tool	Role in this project
Microsoft Fabric portal	Hosts the semantic model, the app item, its SQL database and the deployed app (workspace BI Reports).
Visual Studio Code	The editor where the project folder is opened and the code is reviewed.
Claude Code extension (Anthropic)	AI development assistant inside VS Code: reads the requirement, writes the code and DAX, runs the commands in the project folder, fixes errors and deploys.
Rayfin CLI (Microsoft Fabric Apps)	Scaffolds, connects, builds and deploys the app. It runs through `npx`, so no separate installation is needed.

4.3 Who does what: developer and Claude

Developer	Claude
Creates the app item in the Fabric portal	Scaffolds the project and adds the capabilities
Writes the requirement prompt (Section 5.4)	Designs the app and writes all code, DAX queries and tables
Approves each command Claude wants to run	Runs the commands and reads their output
Completes the Fabric sign-in when the browser opens	Diagnoses and fixes errors, runs tests
Reviews the app and asks for changes	Deploys into the app item and confirms its status

5. Build the app with Claude in VS Code

Five steps take the project from an empty app item to a deployed app. Only Steps 1–4 are done by the developer; in Step 5 Claude does the build while the developer approves and reviews.

5.1 Step 1: create the Fabric App item in the portal

1. Open the **BI Reports** workspace in Microsoft Fabric.
2. Click **+ New item**, search for **Fabric data app** and select it.
3. Enter the name `sales-dashboard-app` and click **Create**. The empty app item appears in the workspace; Claude deploys the app into it in Step 5.

5.2 Step 2: open a project folder in VS Code

1. Create a folder for the project, for example `OneDrive\Fabric App - Interactive Sales dashboard`.
2. In Visual Studio Code choose **File → Open Folder** and select it. Everything Claude creates is written inside this folder.

5.3 Step 3: add the Claude Code extension

1. Open the **Extensions** view, search for **Claude Code** (publisher Anthropic) and click **Install**.
2. Open the Claude panel from the sidebar and sign in with your Claude account.

5.4 Step 4: give Claude the requirement prompt

The whole app is described in one requirement prompt: the setup, the goal, each page, the data and security rules, quality checks and deliverables. Paste it into the Claude panel. The prompt used for this app:

> **Role:** You are building a Microsoft Fabric App for me using the Rayfin framework. Run every command yourself in this VS Code project folder and ask me before running each one.
>
> **Project setup**
>
> I have already created an empty Fabric data app item named `sales-dashboard-app` in the **BI Reports** workspace (New item → Fabric data app). Build the app in code and deploy it into that item.
>
> 1. Create the project code: `npm create @microsoft/rayfin@latest -- "Sales Dashboard App" --template blankapp --workspace "BI Reports"`
> 2. Explore the project structure and the Rayfin framework (the template's capability packs and the installed Rayfin guide).
> 3. Add the capabilities needed: connectors, analytics (semantic model) and data-modeling (app-owned SQL database).
> 4. Connect the existing Power BI semantic model **Interactive sales dashboard** in the **BI Reports** workspace as a read-only connector with delegated authentication, so each user's own permissions apply.
> 5. Validate every DAX query with `npx rayfin connector invoke` before building screens on it.

6. Deploy into the existing item with `npx rayfin up --workspace "BI Reports" --item-name "sales-dashboard-app"` (I will confirm reusing the item), then run locally with `npm run dev` only after that first deployment.

Goal: The existing Power BI report answers "what happened". Build an app on the same semantic model that answers "what will we do about it, and who owns it". It must do things a Power BI report cannot: capture user input, save it, share it with the team and track it through a workflow. Every number taken from the model must match the Power BI report exactly.

App structure: left sidebar with four pages, a year picker (only years with sales), the signed-in user's email and a light/dark theme toggle.

Page 1 – Pulse: KPI tiles (Total Sales and Total Profit with % vs last year, Orders with new customers, Open actions with overdue count); monthly chart of this year vs last year vs the model's monthly target; a "Team plan (next year)" tile that sums the forecasts saved in Plan Studio and compares them with this year's actual.

Page 2 – Plan Studio (main write-back use case: collaborative sales forecasting): grid of Geographic Region × Product Category with the selected year's actual sales from the model as the baseline; each planner enters a growth % per cell (plus Fill all 0/5/10%) and an optional note; the planned value recalculates instantly; **Save draft** writes the planner's cells to an app-owned SQL table and **Submit plan** marks them Submitted; team consensus per cell = average of all planners' plans; a planner activity list (who, cells, total, status, last update). Do not use the word "writeback" in any heading in the app.

Page 3 – Scenario Lab: use each category's real economics from the model (sales, cost, quantity, gross sales, discount); levers for list price (−20% to +20%), discount rate (−10 to +10 points), units sold (−30% to +30%), price elasticity (−3 to 0) and unit cost (−15% to +25%); show projected sales, profit, margin on cost and profit by category; save named scenarios with a note to the app database; a shared library to compare, reload and delete them.

Page 4 – Action Board: detect signals live from the model (countries below last year, loss-making products, highest-value 1–2★ orders with complaint reason); **+ Action** creates a follow-up in the app database with title, impact, priority (High for >15% drop or 1★), assignee and due date (7 or 14 days); already-tracked signals show "Tracked"; Kanban board Open → In progress → Done with edit, delete, manual actions and an "Only mine" filter.

Data and security: three app-owned tables (SalesPlan, Scenario, ActionItem), each row storing the owner's id, email and a timestamp. Every signed-in user can read all rows; only the owner can create, update or delete their own rows, enforced on the server. Never modify the semantic model; keep all DAX in version-controlled `.dax` files.

Design: clean operational look, teal accent, dark-theme ready, honest loading, empty and error states; no sample data.

Quality: typecheck, build, automated tests and lint must pass before deployment; verify write-back by querying the app's SQL database in Fabric.

Deliverables: (1) the deployed app in BI Reports; (2) a Word whitepaper explaining the features, the write-back use case step by step with dark-theme screenshots and proof from the SQL tables, the differentiators from the Power BI

> Interactive Sales Dashboard, every command from connection to deployment, and issues met with their fixes; (3) a short page guide explaining the purpose of each page.

Building your own app? Keep the structure of the prompt and change these values:

What	Value in this project	Change it to
Workspace	BI Reports	Your workspace
App item name	sales-dashboard-app	The item you created in Step 1
Semantic model	Interactive sales dashboard	Your published semantic model
Goal and pages	Pulse, Plan Studio, Scenario Lab, Action Board	Your business scenario and screens
App tables	SalesPlan, Scenario, ActionItem	The data your users need to save

5.5 Step 5: Claude builds, tests and deploys

Claude now works through the build and shows its progress in the panel. The developer's part is small:

- **Approve commands:** Claude asks before each command (for example `npm create …`, `npx rayfin connector add …`, `npx rayfin up …`). You can approve one at a time or allow trusted commands for the session. On the first deployment, also confirm reusing the app item you created in Step 1.
- **Sign in to Fabric:** when `npx rayfin login` opens a browser window, complete the sign-in yourself. Claude never enters credentials.
- **Review and refine:** open the deployed app, check it and ask Claude for changes in plain words. Claude edits the code, re-runs the tests and redeploys.

Follow-up prompts used after the requirement prompt:

#	Prompt (summary)	Outcome
1	Model location: "It is there in BI Reports workspace <URL>"	Connector bound to the BI Reports model by its IDs
2	Error pasted: "Fabric license probe request failed"	Fabric sign-in renewed with `npx rayfin login`
3	Screenshot: "Connector 'salesmodel' is not declared…" in the local app	Claude confirmed local dev does not wire semantic-model connectors; deployed with `rayfin up`
4	"The wordings are going outside the card" / "fix the High part"	Font-size merge fix and no-wrap badges; redeployed
5	"Change 'Turn signals into owners' to 'Turn insights into actions'"	Text updated in the app; redeployed
6	"Where can I check the write-back?"	SQL queries to verify the saved rows in Fabric (Section 8.2)

The result is the deployed app in the `sales-dashboard-app` item (Section 7). Section 6 lists every command Claude ran on the way.

6. Commands Claude ran: from connection to deployment

During Step 5, Claude ran the commands below in the VS Code project folder. They are listed in order so the build can be understood, reviewed or repeated by hand.

6.1 Scaffold the project

```
npm create @microsoft/rayfin@latest -- "Sales Dashboard App" --template blankapp --
workspace "BI Reports"
cd sales-dashboard-app
npx rayfin login status        # Claude asks you to sign in with npx rayfin login if
needed
```

Note on the name: "Sales Dashboard App" is the readable project name. The CLI converts it automatically to lower case with hyphens, `sales-dashboard-app`, for the project folder (hence `cd sales-dashboard-app`) and the Fabric item name, which matches the item created in Step 1 (Section 5.1).

Result: a workspace with `packages/frontend` (React + Vite), `packages/data` (entities), `packages/shared` (contracts) and `rayfin/rayfin.yml` (services: Fabric sign-in, protected static hosting).

6.2 Add capabilities

```
npm run pack:add -- connectors      # connector-aware Rayfin client
npm run pack:add -- analytics       # semantic-model connector + query hook
npm run pack:add -- data-modeling   # turns on the app SQL database (mssql)
```

6.3 Connect the semantic model

```
npx rayfin connector search "Interactive sales dashboard" --type fabric-semanticmodel --
all-workspaces --json
npx rayfin connector add --type fabric-semanticmodel `
  --workspace-id cd7d60eb-cabd-4b8b-af1a-b940f9106c35 `
  --item-id 7cfc0163-47d2-4108-81b5-bfc1f5c9b343 `
  --name salesModel --operations executeQuery --yes --json
npm install -w @rayfin-app/frontend @microsoft/rayfin-connector-fabric-
semanticmodel@1.36.2 @microsoft/rayfin-connectors@1.36.2
```

Search returned two models with the same name (BI Reports and My workspace); the BI Reports one was selected by its IDs. The CLI registered the connector as `salesmodel` in `rayfin.yml` (delegated auth, `executeQuery` only) and generated `packages/frontend/src/lib/connectors.ts`.

6.4 Validate DAX before writing UI

```
npx rayfin connector invoke salesmodel executeQuery --file inspection/p02-by-
year.query.json --json
# 11 discovery probes + 7 app queries (year 2020) - all returned status "success"
```

Each query file is JSON with a `query` property. Probing first fixed the exact result columns (for example `Calender[Year]`, `[Sales]`) that the UI then binds to. Example app query:

```
EVALUATE
SUMMARIZECOLUMNS (
```

```
    'Location Details'[Geographic Region],
    'Products Details'[Products Category],
    TREATAS ( { {{YEAR}} }, 'Calender'[Year] ),
    "Sales", [Total Sales],
    "Profit", [Total Profit]
)
ORDER BY 'Location Details'[Geographic Region], 'Products Details'[Products Category]
```

6.5 Build and test the app

- Entities: `SalesPlan`, `Scenario`, `ActionItem` in `packages/data/src`, registered in `index.ts`, with browser-safe contracts in `packages/shared`. The full `SalesPlan` definition is shown in Section 8.3.
- Queries: seven `.dax` files in `packages/frontend/src/queries/sales` with factory functions that inject the selected year.
- Pages: `Pulse`, `PlanStudio`, `ScenarioLab`, `ActionBoard`; dependency-free SVG charts; theme tokens in `global.css`.

```
npm run typecheck   # tsc -b
npm run build
npm test            # vitest - 7 files, 61 tests passed
npm run lint
```

The first build was blocked by Windows Application Control (native SWC compiler), so the React build plugin was switched to the pure-JavaScript version (see 6.8):

```
npm install -D -w @rayfin-app/frontend @vitejs/plugin-react@latest
```

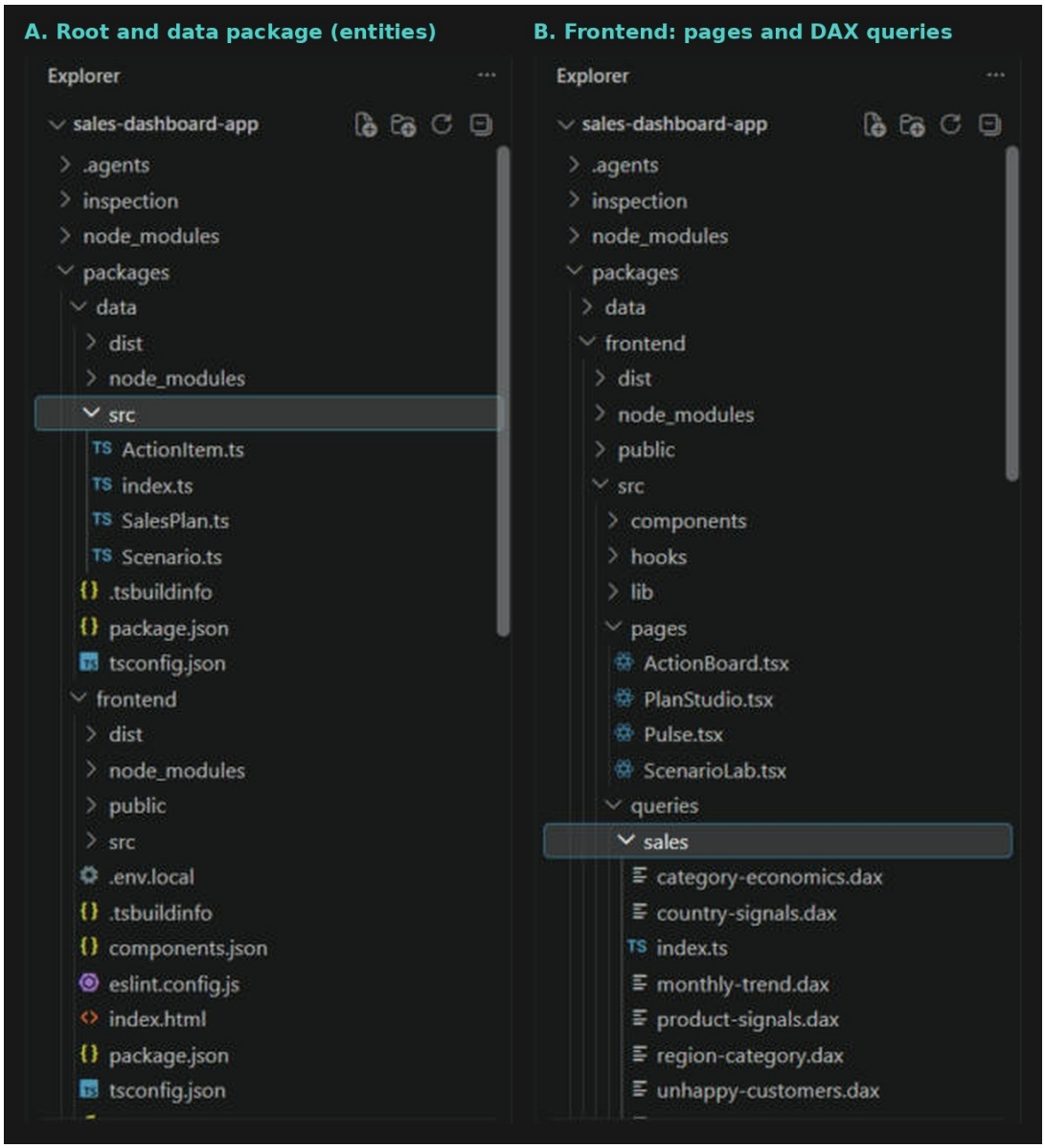


Figure 3. *Project structure in VS Code: (A) the data package with the three entities; (B) the frontend pages and the DAX query files.*

6.6 Deploy into the existing app item

```
npx rayfin login                # Fabric sign-in in the browser (done by the developer)
npx rayfin up --dry-run --workspace "BI Reports" --item-name "sales-dashboard-app"
npx rayfin up --workspace "BI Reports" --item-name "sales-dashboard-app"
npx rayfin up status --json
```

`--item-name` targets the app item created in Step 1 (Section 5.1); the dry run reported *Create or reuse Rayfin item "sales-dashboard-app"*. Because the item was created in the portal and this project had never been deployed there, the first deployment asks you to confirm reusing it. Approve the prompt, or add `--yes` to the command. Note that `--yes` also lets the CLI assign a capacity if the workspace has none, so use it only when that is acceptable. Rayfin records the item in `rayfin/.deployments.json`, so later deployments keep targeting the same item.

Result: the `sales-dashboard-app` item now holds the SQL database `sales-dashboard-app`, the connector configuration and the built frontend on protected static hosting. Later fixes requested in the Claude panel were redeployed with the same command.

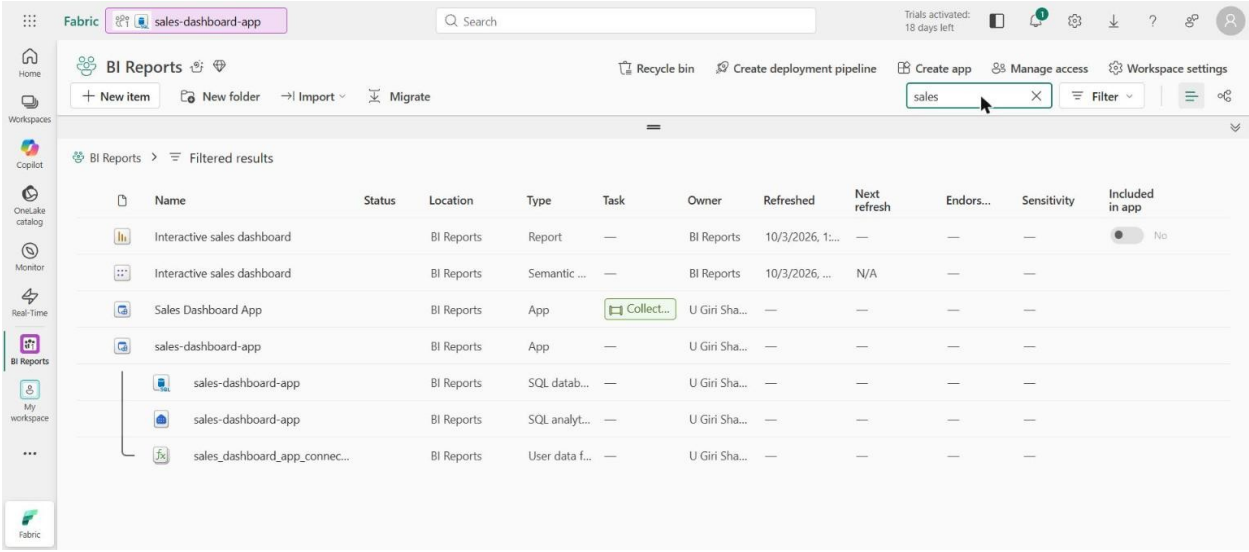


Figure 4. *BI Reports workspace after deployment: the report and semantic model next to the app, its SQL database, SQL analytics endpoint and the connector's user data function.*

6.7 Run locally (optional, after the first deployment)

```
npm run dev          # http://localhost:5173
```

Run the local development server only **after** the first deployment. It then reuses the backend of the `sales-dashboard-app` item. Run before any deployment, it provisions a new Fabric backend on first use and has no option to choose the item, which can leave an extra app item in the workspace. Semantic-model queries run only in the deployed app; locally those areas show a message instead of figures, so model queries are tested with `connector invoke` (6.4).

6.8 Issues Claude met and tips

Issue	Cause	Resolution
Build failed: "An Application Control policy has blocked this file" (swc.win32-x64-msvc.node)	Windows Smart App Control blocked the native SWC compiler used by @vitejs/plugin-react-swc	Switched to the pure-JavaScript `@vitejs/plugin-react` in vite.config.ts and vitest.config.ts
Connector name is `salesmodel`, not `salesModel`	The CLI normalises names to lower case	App code uses the name from rayfin.yml
`rayfin dev`: "Fabric license probe request failed"	Expired or stale sign-in token	`npx rayfin login`, then retry
Local app: "Connector 'salesmodel' is not declared..."	`rayfin dev` parses but does not wire semantic-model connectors	Deploy with `npx rayfin up`

Issue	Cause	Resolution
Labels and badges rendered too large and wrapped	tailwind-merge treated custom size tokens (text-200) as colours and dropped them	`extendTailwindMerge` with the app's font-size scale; badges set to no-wrap
Several models with the same name in `connector search`	Copies of the model in different workspaces	Choose by workspace name and use the explicit workspace and item IDs
DAX fails with odd syntax errors from the terminal	The shell mangles quotes and brackets in inline DAX	Put the query in a JSON file and use `--file`
First deployment stops: item "already exists ... not deployed there before"	The item was created in the portal	Approve the prompt or add `--yes` (6.6)
An extra app item appears in the workspace	`npm run dev` was run before the first deployment and provisioned its own backend	Deploy once with `--item-name` first, then run `npm run dev` (6.7)
Project broken after moving the folder on Windows	Moving the folder follows the npm workspace links in `node_modules`	Create the project in its final, short path; or delete `node_modules` before moving and run `npm install`

6.9 Command summary

The complete sequence on one page:

```
# Scaffold and capabilities
npm create @microsoft/rayfin@latest -- "Sales Dashboard App" --template blankapp --
workspace "BI Reports"
cd sales-dashboard-app
npx rayfin login status
npm run pack:add -- connectors
npm run pack:add -- analytics
npm run pack:add -- data-modeling

# Connect and validate the semantic model
npx rayfin connector search "Interactive sales dashboard" --type fabric-semanticmodel --
all-workspaces --json
npx rayfin connector add --type fabric-semanticmodel --workspace-id <workspace-id> --
item-id <model-id> --name salesModel --operations executeQuery --yes --json
npx rayfin connector invoke salesmodel executeQuery --file inspection/<query>.query.json
--json

# Build and test
npm run typecheck && npm run build && npm test && npm run lint

# Deploy into the existing item (approve item reuse, or add --yes)
npx rayfin login
npx rayfin up --dry-run --workspace "BI Reports" --item-name "sales-dashboard-app"
npx rayfin up --workspace "BI Reports" --item-name "sales-dashboard-app"
npx rayfin up status --json
```

```
# Optional: run locally, only after the first deployment
npm run dev
```

7. Feature walkthrough: the finished app

All app screenshots below were captured from the deployed app in dark theme (toggle at the top right of the app).

7.1 Pulse — live performance vs plan

Pulse opens on the latest year in the model. KPI tiles come from one DAX query over `Calender[Year]`, so the year picker lists only years that actually have sales. The monthly chart overlays this year, last year and the model's monthly target. The **Team plan 2021** tile is not in the model at all: it is the sum of the forecasts planners saved in Plan Studio, compared with the 2020 actual.

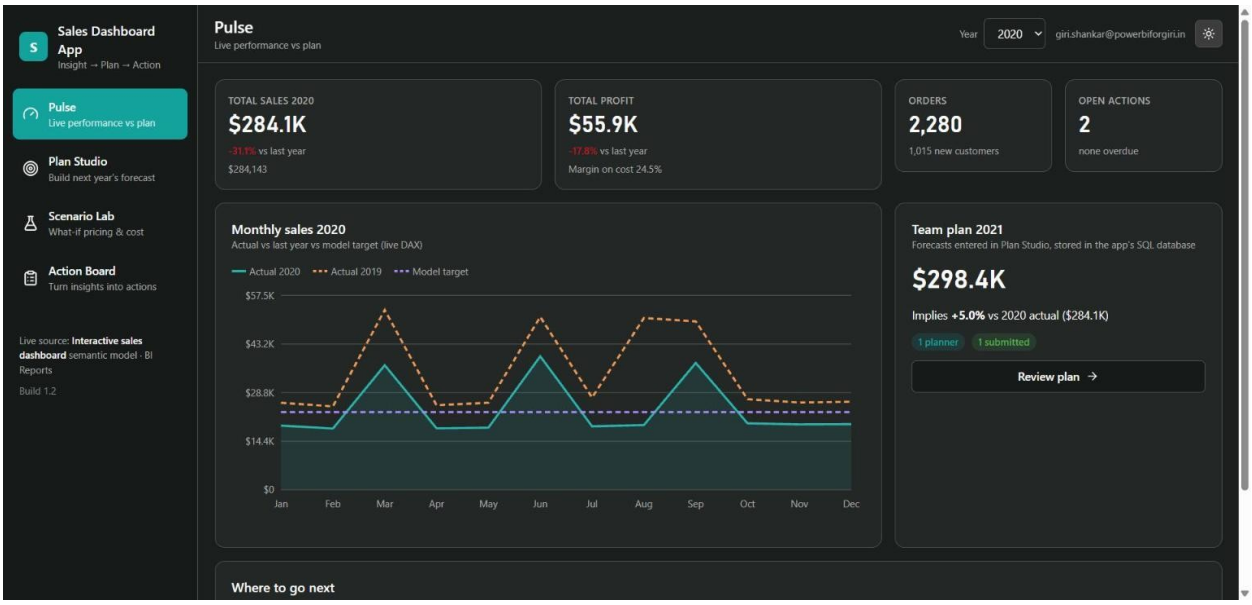


Figure 5. *Pulse (2020). The KPIs match the Power BI report exactly; the Team plan tile comes from data written by users.*

7.2 Plan Studio — collaborative forecast (write-back)

Covered step by step in Section 8.

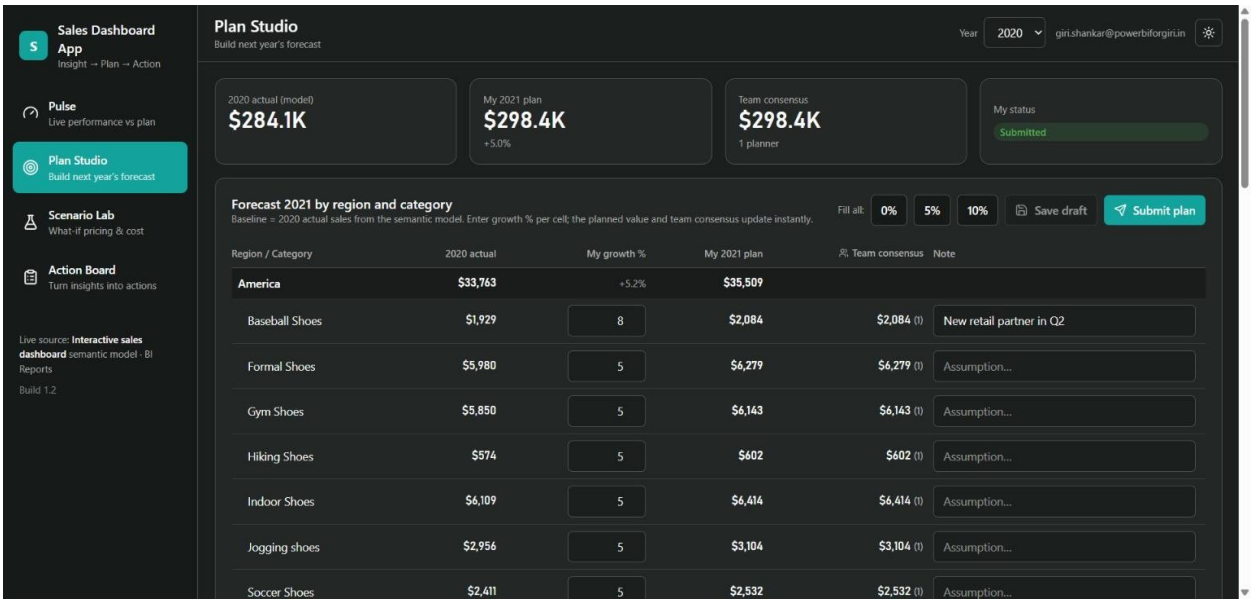


Figure 6. *Plan Studio: 2020 actuals per region and category from the model, each planner's growth %, the resulting 2021 plan and the team consensus.*

7.3 Scenario Lab — what-if on real economics

The levers apply to each product category's real 2020 economics from the model (sales, cost, quantity, gross sales before discount and discount amount). For each category:

- the current discount rate is `discount ÷ gross sales`; the scenario shifts it by the discount lever (percentage points);
- volume changes by the units lever plus `elasticity × price change`;
- projected sales = sales × (1 + price) × volume factor × (1 − new discount) ÷ (1 − current discount);
- projected cost = cost × volume factor × (1 + unit-cost change); profit = sales − cost.

In the example, list price +3%, discount −2 points and elasticity −0.5 lift projected profit by **+27.2%** to $71.2K, and turn Indoor Shoes from a $1.1K loss into a $1.2K profit. Scenarios are saved with a name and note, shared with the team and can be loaded back into the levers.

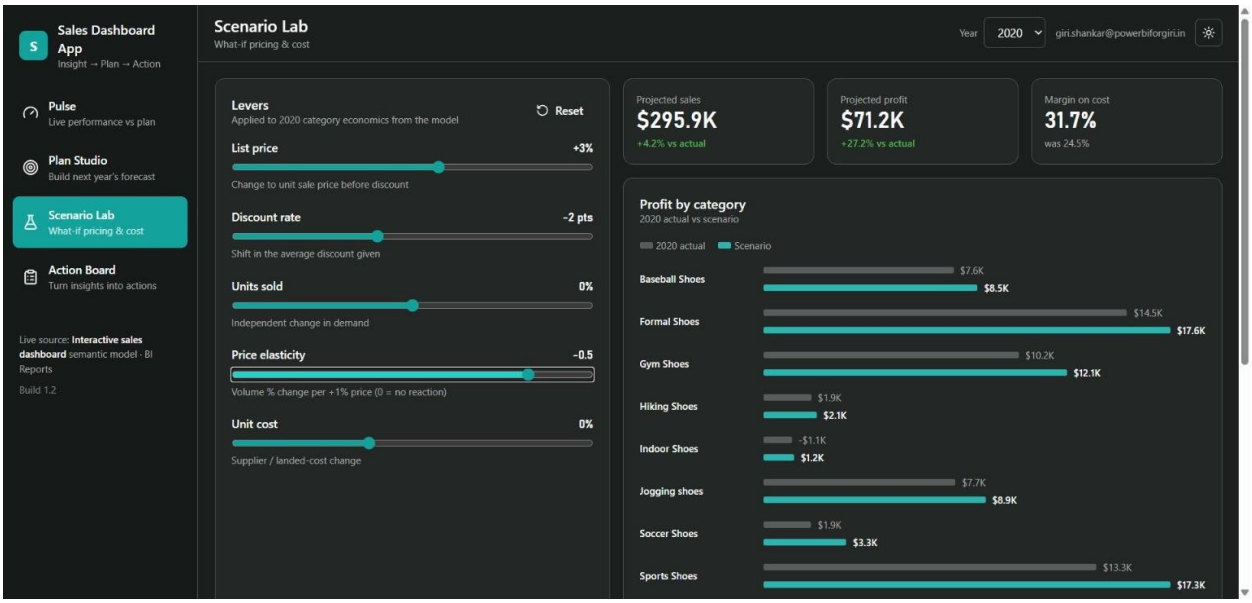


Figure 7. *Scenario Lab: levers on the left; projected sales, profit and margin plus profit by category on the right.*

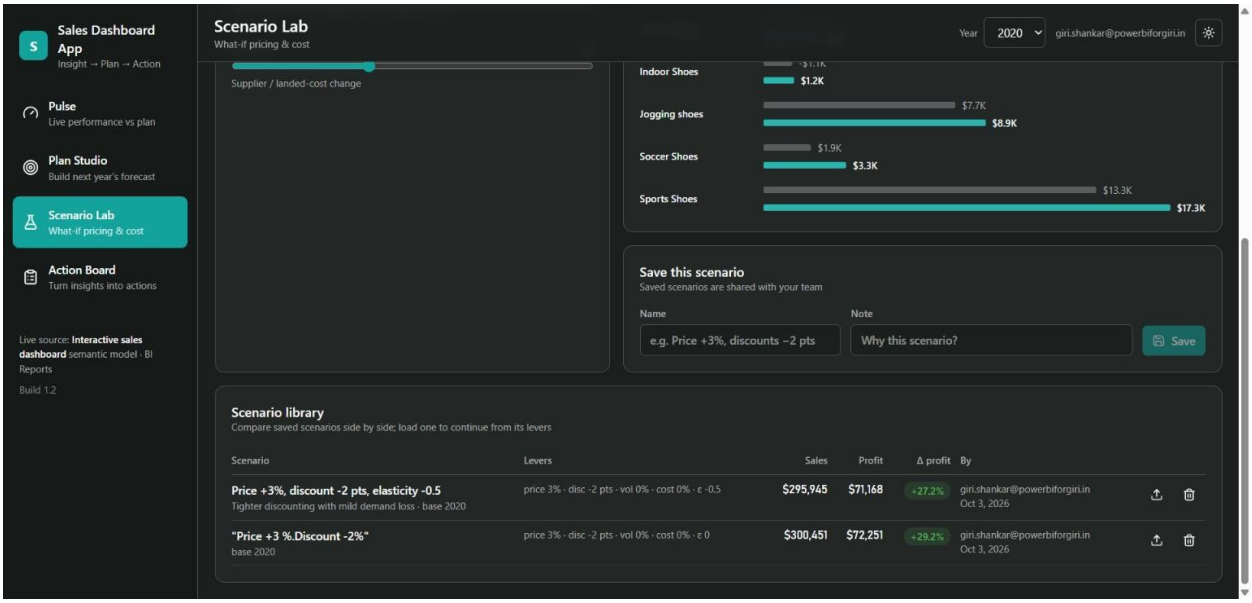


Figure 8. *Scenario library: saved scenarios compared side by side, with who saved them and when.*

7.4 Action Board — turn insights into actions

Three DAX queries look for problems in the selected year: countries whose sales fell versus last year, products with negative profit, and the highest-value orders rated 1–2★ (with the complaint reason). Each signal has a **+ Action** button that creates an action pre-filled with title, impact, priority (High if the drop is over 15% or the rating is 1★), assignee and a due date. Once tracked, a signal shows a **Tracked** badge so the same issue is not raised twice. Cards move across **Open → In progress → Done**, and can be edited (assignee, due date, notes) only by their owner.

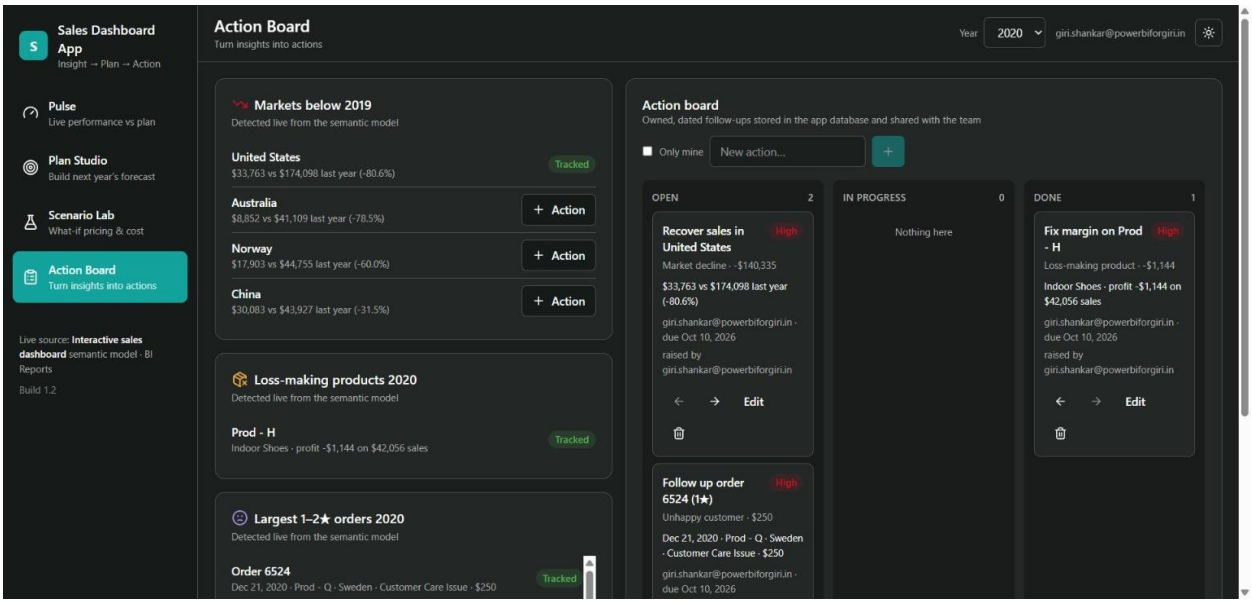


Figure 9. *Action Board: signals detected live from the model (left) and the team's action board (right).*

8. Write-back use case: collaborative sales forecasting

Write-back means the app does not only *read* the semantic model; it lets users *write* new business data, and stores it so that it can be read back, shared, audited and analysed. In this app the write-back target is a **Fabric SQL database created and owned by the Fabric App**. The semantic model is never modified: it stays the governed source of actuals, and the app database holds the plans that people add on top of it.

8.1 Business scenario

Regional and category managers must agree next year's sales plan. Today this is done in spreadsheets emailed around after looking at the Power BI report: numbers go stale, versions conflict and nobody knows who changed what. In the app, every planner starts from the same live actuals, enters a growth % per region × category, and the team consensus updates as soon as anyone saves.

8.2 Step by step

1. Open **Plan Studio** and pick the base year (2020). The grid loads 2020 actual sales per region and category from the semantic model (query `region-category.dax`).
2. Type a growth % for a cell or use **Fill all 0% / 5% / 10%**. The 2021 plan and region subtotals recalculate instantly in the browser; the changed cell is highlighted until saved.
3. Optionally add an assumption in **Note** (for example *Corporate uniform contract*).
4. Click **Save draft**. For each changed cell the app creates the planner's row in `SalesPlans`, or updates it if it already exists, stamping `owner_id`, `ownerEmail` and `updatedAt`.
5. Click **Submit plan** to set every one of the planner's rows to *Submitted*. **Team consensus** is the average of all planners' figures for each cell; Pulse shows the total and how many planners have submitted.
6. Verify in the Fabric SQL database: the same values are in `dbo.SalesPlans`.

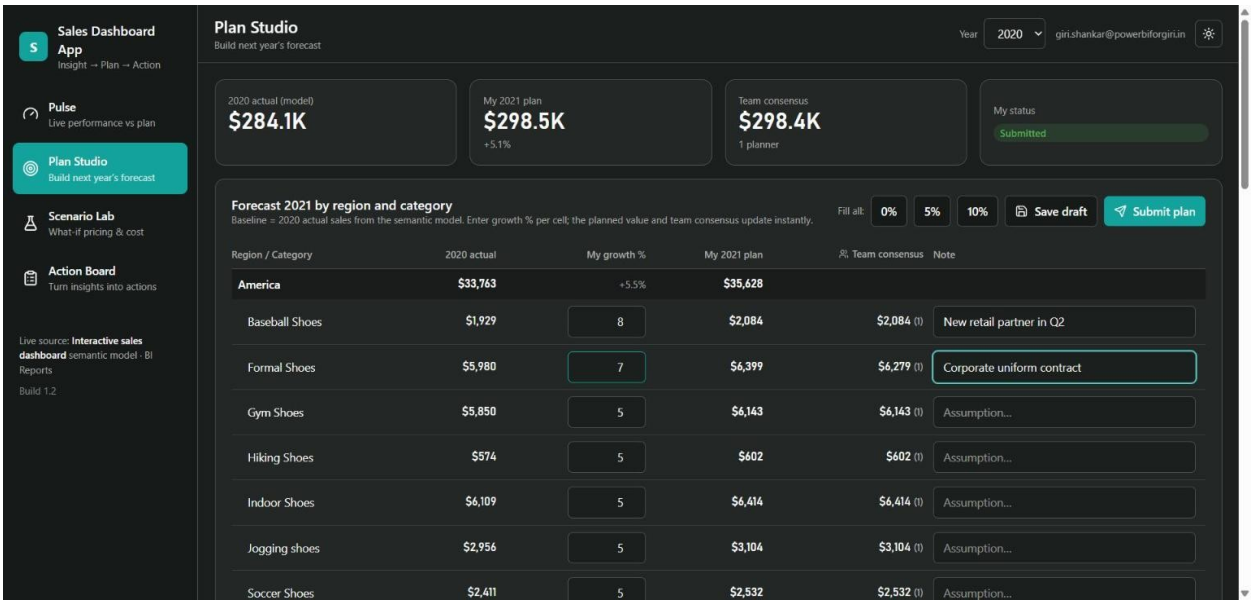


Figure 10. *Steps 2–3: America / Formal Shoes changed from 5% to 7% with a note. The new plan ($6,399) is calculated before saving; the team consensus still shows the saved value ($6,279).*

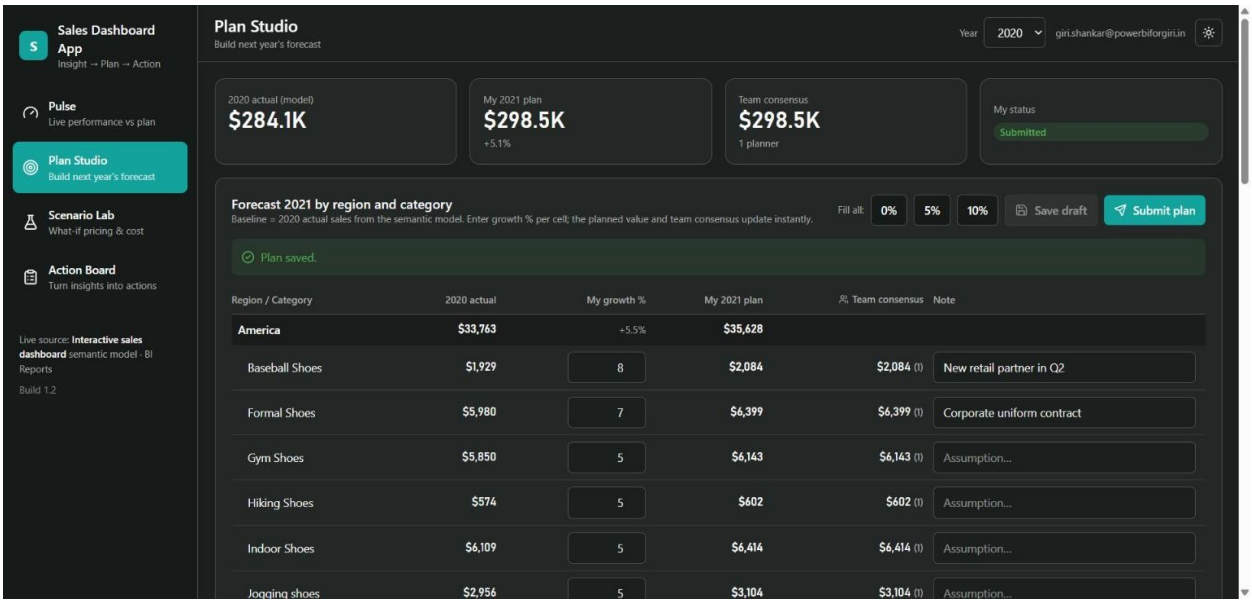


Figure 11. *Step 4: after Save draft the change is stored. "Plan saved." confirms it, and the team consensus updates to $6,399.*

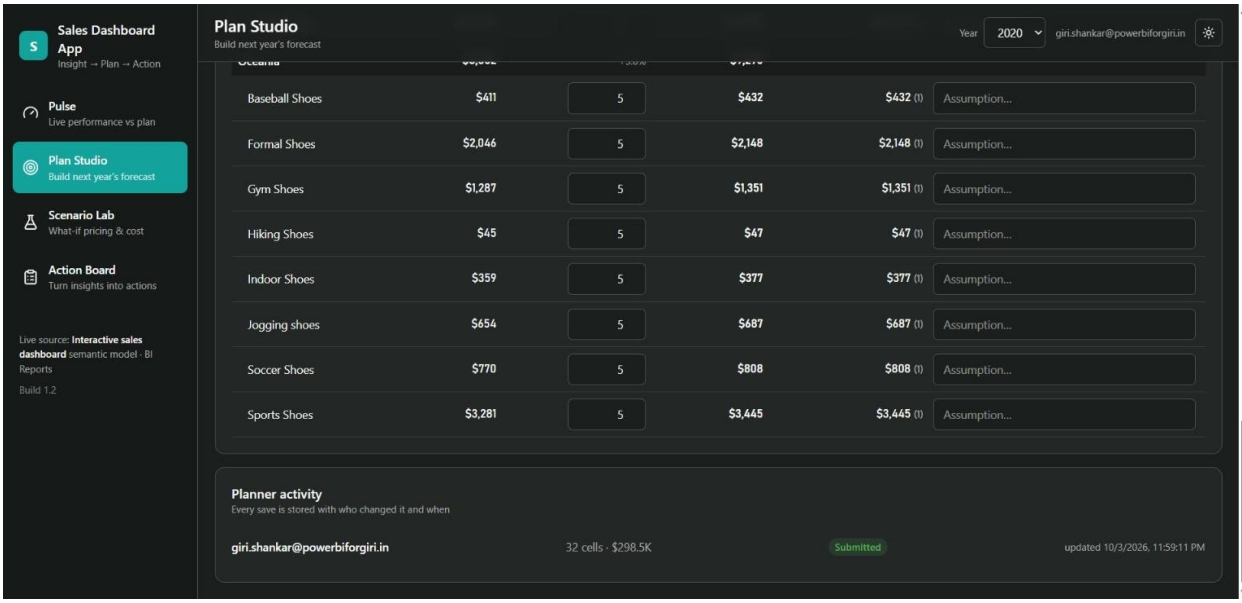


Figure 12. *Step 5: planner activity shows each planner, the number of cells, their total, status and last update.*

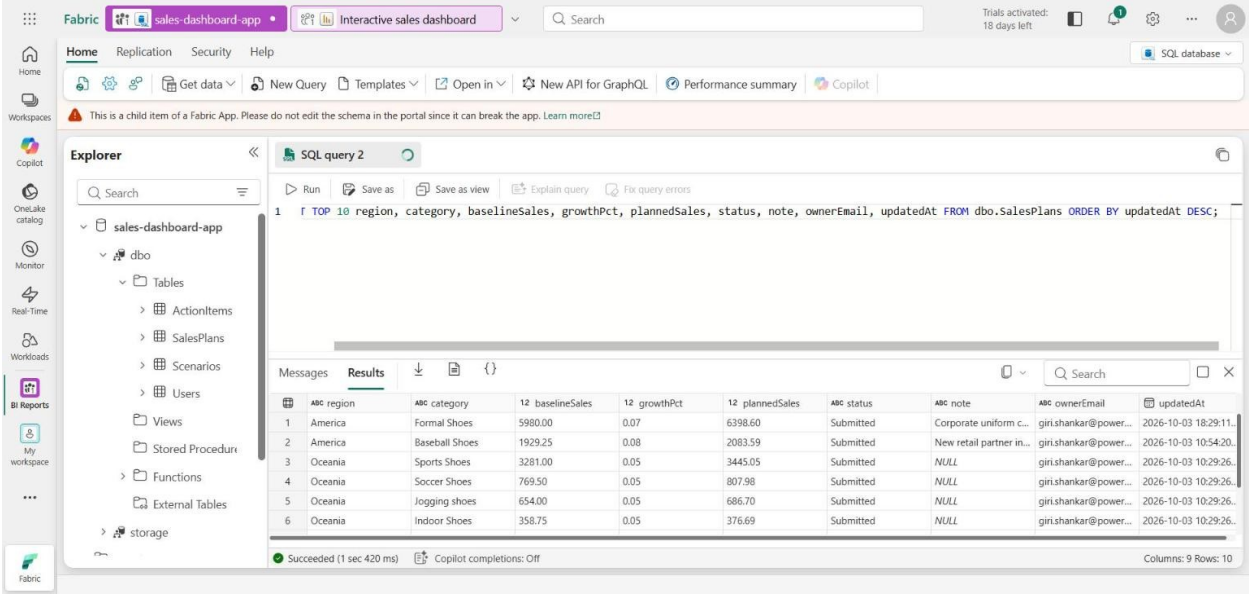


Figure 13. *Step 6: the same row in the Fabric SQL database (dbo.SalesPlans): growthPct 0.07, plannedSales 6398.60, the note, owner email and timestamp.*

8.3 Data model

> **The SQL database is not created from the semantic model**
>
> The app uses two separate data stores. The semantic model is read only and is never copied or changed. The SQL database is created by Rayfin from the app's own table definitions and starts empty; it fills only with what users save. Model figures enter it only as context saved with a user's input, for example the baseline actual stored next to each plan in `baselineSales`.

	Semantic model (Interactive sales dashboard)	App SQL database (sales-dashboard-app)
What it holds	Historical sales data (2017–2020) from the dataset	Only what users enter in the app: plans, scenarios, actions
How it was created	Existing Power BI model, published earlier	Created by Rayfin from the app's entity definitions (`data-modeling` pack, applied on `rayfin dev` / `rayfin up`)
Access from the app	Read only (DAX queries)	Read and write

Entities are TypeScript classes in `packages/data/src`. On `rayfin up` the CLI turns them into SQL tables plus a GraphQL API with permissions (Data API Builder).

```
@entity()
@role('authenticated', 'read')
@role('authenticated', ['create', 'update', 'delete'], {
  policy: (claims, item) => claims.sub.eq(item.owner_id),
})
```

```
export class SalesPlan {
  @uuid() id!: string;
  @int() planYear!: number;          @int() baseYear!: number;
  @text({ min: 1, max: 100 }) region!: string;
  @text({ min: 1, max: 100 }) category!: string;
  @decimal() baselineSales!: number; @decimal() plannedSales!: number;
  @decimal() growthPct!: number;
  @text({ max: 20, default: 'Draft' }) status!: string;
  @text({ optional: true, max: 500 }) note?: string;
  @text({ max: 200 }) owner_id!: string;  @text({ max: 200 }) ownerEmail!: string;
  @date() updatedAt!: Date;
}
```

Table	Written when	Key fields
dbo.SalesPlans	Save draft / Submit plan	planYear, region, category, baselineSales, growthPct, plannedSales, status, note, owner
dbo.Scenarios	Save in Scenario Lab	name, lever values, baseline and projected sales and profit, note, owner
dbo.ActionItems	+ Action, move, edit, delete	title, signalType, dimensionValue, impact, priority, status, assignee, dueDate, notes, owner

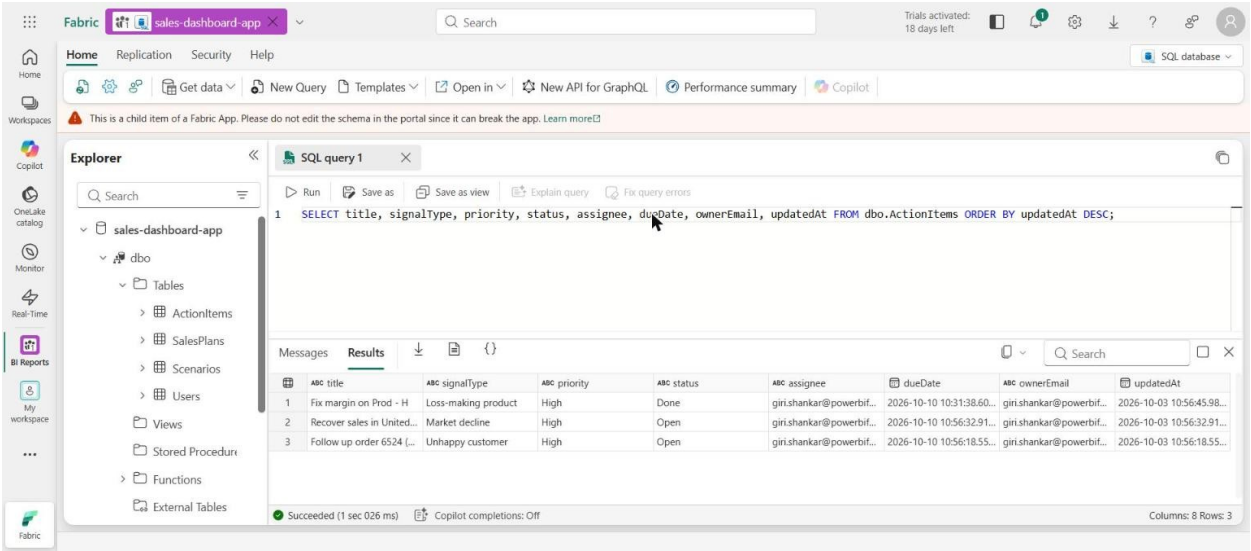


Figure 14. *Action Board write-back in dbo.ActionItems: "Fix margin on Prod - H" moved to Done, two new Open actions.*

8.4 Security and governance

- **Who can write:** only signed-in users. The server-side policy `claims.sub == owner_id` means a user can change or delete only their own rows; it is enforced by the data API, not just the user interface.
- **Who can read:** every signed-in user can read all plans, scenarios and actions, which is what makes consensus and a shared board possible.
- **Source data protection:** the semantic model connection is read-only (`executeQuery` only) and delegated, so model permissions and row-level security still apply per user.

- **Audit trail:** every row carries owner email and timestamp; the SQL database can be queried, mirrored to OneLake or used as a source for further Power BI reporting.
- **Input validation:** text lengths are bounded in the schema (for example a note is at most 500 characters) and mirrored in the form controls.

9. Validation

Check	Result
Model numbers match the Power BI report (2020)	Sales $284.1K, profit $55.9K, −31.1% YoY, 2,280 customers, 1,015 new: identical (Figures 1 and 5)
All DAX queries via `rayfin connector invoke`	18 / 18 successful
TypeScript typecheck, build, lint	Pass
Automated tests (vitest)	61 / 61 pass, including authentication-gate tests
Write-back: Plan Studio	Save and Submit persisted; visible in dbo.SalesPlans (Figure 13)
Write-back: Scenario Lab	Two scenarios saved and listed in the library (Figure 8)
Write-back: Action Board	Create, move to Done and owner stamping persisted in dbo.ActionItems (Figure 14)
Deployment	`rayfin up status`: deployed, authenticated, endpoint reachable

10. Differentiators vs the Power BI Interactive Sales Dashboard

Both read the same semantic model, so they agree on every number (compare Figure 1 with Figure 5). The difference is what users can *do*.

Capability	Power BI report	Sales Dashboard App
Write-back of business data	Not supported natively; needs Power Apps visuals, translytical task flows or other add-ons	Built in: plans, scenarios and actions saved to an app-owned Fabric SQL database
Multi-user collaboration	Everyone views the same visuals; no shared input	Each planner's inputs are stored; team consensus computed from everyone's plans
Workflow and status	None	Draft → Submitted plans; Open → In progress → Done actions
What-if analysis	What-if parameters, not saved, not shared	Five-lever model on real category economics; scenarios named, saved, compared and reloaded
Insights → actions	Conditional colours highlight problems	Signals detected by DAX become owned, dated, prioritised actions; tracked signals are de-duplicated
Audit trail	Not available for user decisions	Owner and timestamp on every row; queryable in SQL
Per-row security on user input	Not applicable	Server-side policy: users edit only their own rows
User interface	Fixed visual canvas	Full React application: forms, inline editing, Kanban board, keyboard-accessible controls, light and dark themes
Query control	Generated by visuals	Hand-written, version-controlled DAX files (`.dax`) validated with `rayfin connector invoke`
Delivery	Publish from Power BI Desktop	Code in Git; build, test and deploy with `npx rayfin up` (CI/CD ready)
Reuse of existing assets	—	Reuses the existing semantic model and measures; no duplication of business logic

> **Positioning**
>
> The app does not replace the report. The Power BI report remains the best tool for exploring the data; the app is where the team turns what it sees there into committed plans and owned actions.

11. Operating and extending the app

11.1 Day-to-day use

- Open the app from the BI Reports workspace (item `sales-dashboard-app`) or at its hosting URL; users sign in with their organisational account.
- When the semantic model refreshes, the app shows the new actuals immediately (there is no app-side cache).
- To check what users entered, query `dbo.SalesPlans`, `dbo.Scenarios` and `dbo.ActionItems` in the app's SQL database (read-only queries; do not change the schema in the portal).
- To change the app: open the project folder in VS Code and describe the change to Claude. Claude edits the code, runs `npm run typecheck && npm test`, then redeploys with `npx rayfin up --workspace "BI Reports" --item-name "sales-dashboard-app"`.

11.2 Future enhancements

The app is a first version. These features can be added later with more code:

- **Plan vs actual in Power BI:** a report page that reads the app's SQL database, so managers see saved plans next to actuals in Power BI.
- **Approval step:** extend the plan workflow from Draft → Submitted to Draft → Submitted → Approved, reviewed by a manager.
- **Questions in plain English:** connect a Fabric data agent so users can ask questions about the sales data without DAX.
- **Test before release (Dev → Prod):** keep a test copy for checking changes and a live copy for users.
- **Weighted team consensus:** let senior planners' figures count more than a simple average.
- **Monthly plan:** split each yearly plan across months using the monthly target pattern in the semantic model.
- **Manager role for actions:** let a manager update or reassign any action, not only its creator.

12. Advantages and technical limitations of the Fabric App approach

The advantages are based on what was observed while building, testing and deploying the Sales Dashboard App. The technical limitations apply to Fabric Apps built with Rayfin in general.

12.1 Advantages

Advantage	What it means in practice
One governed source of truth	The app queries the existing semantic model and reuses its measures (Total Sales, Sales LY, Total Profit % and others). Nothing is copied or re-modelled; the app's 2020 figures matched the Power BI report exactly.
Write-back without extra products	The Fabric App comes with its own SQL database. Plans, scenarios and actions are stored there immediately, in the same workspace as the report and model, with no Power Apps, flows, external database or separate hosting.
Security carried through	The semantic model is queried with each user's own identity, so existing permissions and row-level security apply. For app-owned data, access rules are declared with the table definition and enforced on the server.
Single sign-on and protected hosting	Users sign in with their organisational account, inside the Fabric portal or in a browser; the hosted app stays private and asks for sign-in when a session expires.
Freedom in user interface and workflow	A Fabric App is a full web application, so the interface is not limited to a report canvas. It can include editable grids, forms, sliders, Kanban boards, approval steps and light/dark themes, as well as animated transitions, animated charts and interactive 3D visuals built with standard web libraries.
Engineering discipline	All code, including DAX, is plain text that can be version-controlled and reviewed. Each query was validated before use, and type checks, 61 unit tests and linting ran before every deployment.
Simple, repeatable deployment	One command (`npx rayfin up`) builds the frontend, applies database tables and connector settings and publishes the app; a dry run previews changes and a status command confirms the result.
Everything stays inside Fabric	The app, its SQL database, SQL analytics endpoint and connector function sit next to the report and model. Stored plans can be queried in SQL and later reported on in Power BI.
Extensible	The same project can add more connectors (warehouse, SQL database, Lakehouse), server-side functions for trusted logic or external services, and further tables without starting again.

12.2 Technical limitations

These limitations apply to Fabric Apps built with Rayfin in general.

Limitation	What it means
Model queries need a deployed app	The local development server does not run semantic-model queries, so screens that use model data can only be fully tested after deployment.
Semantic model is read-only from the app	The semantic-model connector only runs queries. User input cannot be written back into the model; it can only be stored in the app's own SQL database.
Code-only development	There is no drag-and-drop designer. Queries, charts and pages are written as code, so TypeScript, React and DAX skills are required. Queries depend on exact table, column and measure names, so renaming a measure in the model breaks the app until the code is updated.
Data API constraints	Reads return one page of rows at a time (100 by default). There is no built-in record count, text fields need an explicit maximum length, and many-to-many relationships need a separate linking table.
Limited permission model	The platform's built-in access rules know only two roles: anonymous and signed-in. Custom roles such as manager or approver are not built in; they must be implemented in the app's own code.
Schema ownership	The SQL database belongs to the app, and its tables cannot safely be edited in the Fabric portal. Table changes are made in code and applied on deployment; changes that remove data need an explicit override.
Early-release framework	Rayfin is in early release, so commands and behaviour may change between versions.
Prerequisites and sign-in	A Fabric capacity with the Fabric Apps workload, a semantic model published to the service (a local .pbix cannot be used) and Contributor rights are required. CLI sign-in sessions expire and must be renewed.

13. Conclusion

- **Builds on existing semantic models:** a Fabric App can query an organisation's published semantic models directly, reusing the same measures and business logic so its figures stay consistent with Power BI reports.
- **Adds write-back natively:** each app can have its own Fabric SQL database, so users can enter, update and track data such as plans, approvals, comments or tasks without separate tools or hosting.
- **Turns analytics into applications:** a full web front end supports forms, workflows, what-if models and collaboration that go beyond a report canvas.
- **Governed and secure by default:** organisational sign-in, the user's own model permissions and row-level security, server-enforced data rules, and everything kept inside the Fabric workspace alongside reports and models.
- **Complements Power BI rather than replacing it:** reports remain best for self-service exploration, while Fabric Apps suit scenarios where people must act on data. The trade-offs are the developer skills it requires (code and DAX) and a framework that is still in early release.

The approach followed in this project: the Sales Dashboard App combines existing data with user input on the same page. Sales, profit and last-year figures are read live from the *Interactive sales dashboard* semantic model and are never changed. The plans, scenarios and actions that users enter are saved to the app's own SQL database. Each page shows the two side by side: Plan Studio, for example, shows actuals next to each planner's growth % and the team average. A Power BI report can only read data; this app reads it, captures decisions against it and shows both together.

14. References and acknowledgements

14.1 Dataset credit

The sales data in the **Interactive sales dashboard** semantic model comes from the practice workbook **Sales Dataset.xlsx** (sheets: Sales Data, Sales Target, Sales Channel Details, Channel Type Details, Location Details, Products Details, Discount Details, Complaints & Returns). It was provided as course material in a Udemy Power BI course (*40 hrs premium content*, practice files dated 09-10-2022, exercise *Interactive Sales Dashboard - 1*). The dataset and the original report exercise belong to their respective course authors and are used here for learning and demonstration only.

All business figures in this document (for example 2020 sales of $284,143) are computed from that practice dataset and do not describe a real company.

14.2 Original work in this project

- The Sales Dashboard App design: the four sections, the collaborative forecasting (write-back) use case, the scenario model and the insights-to-actions board.
- All application code, DAX queries, entity definitions, screenshots and the architecture diagram in this document.
- The project scaffold, authentication flow and capability packs come from Microsoft's Rayfin `blankapp` template and CLI, used under their MIT licence.

14.3 Technical references

- Microsoft Learn: Create and deploy your first Fabric app with the Rayfin CLI. https://learn.microsoft.com/fabric/apps/create-app-with-cli
- Microsoft Learn: Rayfin CLI reference. https://learn.microsoft.com/fabric/apps/cli-reference
- Microsoft Learn: Create an app connected to a semantic model. https://learn.microsoft.com/fabric/apps/data-apps-template
- Microsoft Learn: Fabric Apps project structure and data models. https://learn.microsoft.com/fabric/apps/project-structure · https://learn.microsoft.com/fabric/apps/data-models
- Rayfin documentation: Connectors. https://rayfin.ai/docs/connectors
- Rayfin guide (installed with the CLI, version 1.36.2): data/graphql.md, data/permissions.md, cli/connectors/category-b-function-bridge.md, known-limitations.md.

Appendix A. Reference identifiers

Item	Value
Workspace (BI Reports)	cd7d60eb-cabd-4b8b-af1a-b940f9106c35
Semantic model (Interactive sales dashboard)	7cfc0163-47d2-4108-81b5-bfc1f5c9b343
Fabric App item (sales-dashboard-app)	89599266-9395-4546-87f4-808a4d2da33b
SQL database (sales-dashboard-app)	c8afda3c-0496-4cbf-b74d-3e16ce7d47b7
Connector	salesmodel · fabric-semanticmodel · delegated · executeQuery
App URL	https://great-twist-27fbb5fef0-centralindia.webapp.fabricapps.net
Rayfin CLI	1.36.2

Appendix B. Project structure

```
sales-dashboard-app/
  rayfin/rayfin.yml                services, connector salesmodel
  rayfin/connectors/salesmodel/    generated connector schema
  packages/data/src/               SalesPlan.ts, Scenario.ts, ActionItem.ts, index.ts
  packages/shared/src/index.ts     record contracts + UniversalAppSchema
  packages/frontend/src/
    App.tsx                        shell, navigation, year picker, theme
    pages/                         Pulse, PlanStudio, ScenarioLab, ActionBoard
    queries/sales/*.dax            7 DAX queries + factories (index.ts)
    lib/model.ts                   DAX result -> rows; lib/app-data.ts: SQL CRUD
    components/                    ui.tsx, charts.tsx
  inspection/*.query.json          DAX validation payloads
```

See Figure 3 for the same structure in VS Code.

Appendix C. DAX queries used by the app

File	Returns
years.dax	Sales, cost, profit, profit %, sales LY, profit LY, customers, new customers, ratings by year
monthly-trend.dax	Sales, profit, sales LY, target by month for the selected year
region-category.dax	Sales and profit by region × category (plan baseline)
category-economics.dax	Sales, cost, profit, quantity, gross sales, discount by category (scenario baseline)
country-signals.dax	Countries with sales below last year
product-signals.dax	Top 10 products with negative profit
unhappy-customers.dax	12 highest-value orders rated 1–2★ with complaint reason